Performance Testing Report Submission Template

Computer Systems and Network Engineering - Final Year Research Project

Project Title: **A Lightweight Trust Management Framework in the Networking Layer of IoT Devices**

Student Names: **Wijekoon H M R K**

Student IDs: **IT22065162**

Supervisor: **Ms. Shashika Lokuliyana, Mr. Anuradha Jayakody**

Date of Submission: **31st March 2026**

1. Introduction and Objective

This report focuses specifically on the Trust & Security-Aware RPL Routing Mechanism, a custom Objective Function (TS-OF, OCP=0xAA) implemented within the Contiki-NG IoT operating system and evaluated using the Cooja network simulator. The broader group framework consists of four tightly integrated modules: the Trust Metric Engine, the IDS & Attack Detection module, the Consensus & Isolation module, and the TS-OF Routing mechanism.

The RPL protocol (RFC 6550) is the de facto standard routing protocol for resource-constrained IoT networks. It organizes devices into a Destination-Oriented Directed Acyclic Graph (DODAG) and selects parent nodes using an Objective Function (OF). The standard Minimum Rank with Hysteresis Objective Function (MRHOF, OCP=1) selects parents based solely on ETX (Expected Transmission Count). This leaves the network vulnerable to sinkhole attacks where a malicious node advertises falsely low ETX values to attract traffic and selective forwarding attacks, where a node silently drops a fraction of packets. Existing trust-aware RPL extensions in the literature are often computationally expensive and impractical for constrained IoT devices.

The TS-OF mechanism addresses this gap by replacing MRHOF with a composite Objective Function that integrates three factors into every routing decision: raw link quality (ETX), a per-neighbor trust score, and a security factor penalty. The composite formula is intentionally lightweight — one floating-point computation per parent evaluation — with no machine learning, no blockchain, and no heavy cryptographic operations. It is designed to be tunable through three weight parameters defined at compile time.

2. Performance Testing Environment Setup

Hardware Configuration:

All simulation experiments were conducted on personal computers running Oracle VirtualBox with an Ubuntu virtual machine, as Contiki-NG requires a Linux environment. All simulations used Cooja Motes (full software simulation).

Host Machine	Personal computer (Windows), Oracle VirtualBox host for Ubuntu VM
RAM	16GB
Storage	512GB SSD
Virtualization	Oracle VirtualBox, host Ubuntu VM running Contiki-NG
Simulated Nodes	6 Cooja Motes (IDs 1-6)
Node Roles (6-node demo)	Node 1: DODAG root Node 2: Sinkhole attacker Node 3: Selective Forwarding attacker Node 4: Faulty Hardware Node 5-6: Normal sensor nodes

Software Configuration:

OS (VM)	Ubuntu LTS, inside oracle VirtualBox on Windows host
Contiki-NG - v5.1-21-g9ddc95f30-dirty	provides RPL-Lite, 6LoWPAN, 802.15.4 MAC
Cooja Simulator	Launched via: ant run from ~/Contiki-ng/tools/cooja. Provide mote level energy models, traffic visualization, mote output logging, and attack scenario support
RPL Configuration	RPL Lite in non storing mode Baseline – standard RPL Objective function (MRHOF) OCP=1 Implemented – Trust and Security aware Objective function (TS-OF) OCP=0xAA
Development language	C language (all protocol stack modification)
Analysis language	Python language (post simulation log processing, CSV parsing, statistical analysis)
Version control	GitHub branch – Integrated new
Project Directory	~/Contiki-ng/examples/ts-rpl-demo/

Network Topology and Condition

The 6-node simulation used a deliberate linear chain topology to maximize the observation impact of routing attacks. In this chain data flows upward toward the root.

```
Simulation Topology (data flows upward toward root):

  Node 6 --> Node 5 --> Node 4 --> Node 3 --> Node 2 --> Node 1 (Root)
                                                          fd00::201:1:1:1


  Node 1  :  DODAG root (Rank 0)
  Node 2  :  Sinkhole attacker      -- attack begins after 120s warm-up period
  Node 3  :  Selective Forwarding attacker
  Node 4  :  Faulty hardware node   -- overheating (~65 degrees C, T_TH ~= 0.19)
  Node 5  :  Normal sensor node
  Node 6  :  Normal sensor node
```

3. Tools and Frameworks Used

Contiki-NG and Cooja

Contiki-NG (v5.1-21-g9ddc95f30-dirty) is the primary development and simulation platform. It provides accurate RPL-Lite implementation, the 6LoWPAN network stack, IEEE 802.15.4 MAC layer, and the Energest energy accounting module (used by the Trust Metric Engine for per-node residual energy estimation). The Cooja simulator, launched via ant run from ~/contiki-ng/tools/cooja, provides the controlled simulation environment with Cooja Mote support, mote output logging, and customisable attack scenarios. All 6 nodes boot within 1 second of simulation start and join the DODAG within 18 seconds.

TS-OF Specific files

File	Contribution and role in testing
rpl-ts-of.h	Header file declaring the rpl_ts_of vtable. Single extern declaration. Sets OCP=0xAA (170). Included by project-conf.h to register TS-OF with the RPL runtime.
rpl-ts-of.c	Core TS-OF implementation. Implements all 8 RPL-Lite Objective Function callbacks. The nbr_link_metric() callback is the critical function it reads ETX from link_stats, fetches trust and secf from ts-metrics, and computes the composite rank. Returns RPL_INFINITE_RANK for blacklisted nodes.
ts-metrics.h / .c	Per-neighbour in-memory trust/security/ETX table (32 slots). The shared internal API bridge between all framework modules and the routing component. Default values: trust=0.80, secf=1.00, ETX=1.00. TS-OF reads from this table on every parent evaluation.
ts-rpl-demo.c	Main Contiki-NG process. Root initialisation (Node 1 only), periodic UDP send every 10s, reception and latency measurement at root, ts_parent_switch_cb() registered via RPL_CALLBACK_PARENT_SWITCH, and all TSLOG structured event logging.
project-conf.h	Compile-time configuration: OCP=0xAA, composite weight parameters (α=1.0, β=0.5, γ=1.0), RPL_CALLBACK_PARENT_SWITCH macro, RPL_CONF_SUPPORTED_OFS list, Non-Storing MOP.
Makefile	Adds ts-metrics.c and rpl-ts-of.c to PROJECT_SOURCEFILES. Sets PROJECT_CONF_PATH=project-conf.h. ts-rpl-demo.c is auto-included as CONTIKI_PROJECT.
positions.dat	Cooja node mobility script. Moves Node 2 from (80,50) to (200,200) at t=5s to test link degradation and dynamic parent re-selection behaviour of TS-OF.

Composite Rank Formula

Replaces MRHOF's ETX-only metric with a three-factor composite computed inside nbr_link_metric():

```
Composite = (alpha * ETX) + (beta * (1 - trust)) + (gamma * ETX * (secf - 1))

Where:
  alpha = 1.0   (ETX weight       -- defined in project-conf.h as TS_ALPHA_ETX)
  beta  = 0.5   (Trust penalty    -- defined in project-conf.h as TS_BETA_TRUST)
  gamma = 1.0   (Security penalty -- defined in project-conf.h as TS_GAMMA_SECURITY)

  ETX    : raw link quality from Contiki-NG link_stats_from_lladdr() (float)
  trust  : per-neighbour trust score from ts_get_trust()  [0.0 = untrusted, 1.0 = trusted]
  secf   : per-neighbour security factor from ts_get_security_factor()  [>= 1.0]

All three weight parameters are compile-time tunable via project-conf.h.
```

The formula is structured so that:

1) at healthy defaults (trust=0.80, secf=1.00) the composite is slightly larger than raw ETX, producing uniformly higher ranks compared to MRHOF — this is expected and by design
2) A reduced trust score adds a positive penalty term $\beta\times(1-\text{trust})$ that makes the node less preferred as a parent
3) A raised security factor (secf>1.0) multiplies with ETX to produce a large additive penalty, making the attacker node deeply unattractive
4) for confirmed blacklisted nodes, the routing function returns PL_INFINITE_RANK immediately, bypassing the formula entirely

4. Testing Methodology and Execution

Test Scenarios

Scenario	Description
MRHOF Baseline	Standard RPL with MRHOF (OCP = 1). 6 node topology, no attackers,120s warmup observed. Established reference rank values, PDR, latency, and join time for direct TS-OF comparison
TS-OF Normal	TS-OF active (OCP = 0xAA), all nodes healthy, no attackers, trust=0.80, secf=1.00 at default. Validate OCP=170 in DIO message, composite rank values, parent switch logging and confirm PDR=100% maintain despite higher rank
Sinkhole attack	Node 2 broadcast falsely low ETX, IDS raises PTI alert, ts_of_raise_alert() feeds trust=0.10, secf=2.50 into ts-metrics. Evaluate TS-OF soft penalty (25× cost increase) and hard block (RPL_INFINITE_RANK) following consensus blacklisting; measure time-to-rerouting
SFA attack	Node 3 silently drops packets; IDS raises CDFR alert; penalty values written to ts-metrics for Node 3. Evaluate TS-OF independent isolation of second attacker through same composite penalty mechanism
Simultaneous attacks	Both Node 2 (sinkhole) and Node 3 (SFA) attacking; Severity=2 from IDS; both attacker nodes penalised in ts-metrics simultaneously. Primary integration validation confirm TS-OF correctly handles two concurrent attacker penalties and enforces RPL_INFINITE_RANK for both after consensus
Mobile Topology	S5 conditions with Node 2 mobility (positions.dat). ETX degrades to ~7.44; 20+ parent changes. Validate TS-OF parent re-evaluation behaviour under degraded link conditions; confirm FP=0 maintained when PTI_dynamic threshold adapts to mobile noise

TS-OF Execution Flow Within the Integrated Framework

- Passive collection (ongoing): udp_rx_callback() collects per-packet metrics (inter-packet timing, sequence numbers, data packet counts) for the IDS module. This data indirectly determines whether trust/secf penalty values will be written to the ts-metrics table that TS-OF reads from.
- Trust Engine executes (start of each window): Computes T_FINAL per node using: T_FINAL = (0.25×T_HO) + (0.20×T_ER) + (0.30×T_TA) + (0.25×T_TH). For the faulty hardware node (Node 4), ts_set_trust() writes the degraded

T_FINAL directly into the ts-metrics table — the same table TS-OF reads on every parent evaluation.

- IDS executes (same window, immediately after Trust Engine): Evaluates PTI, CDFR, and SIS flags. On confirmed attack (Severity ≥ 1), ts_of_raise_alert() sets trust=0.10 and secf=2.50 for the attacker node in the ts-metrics table.
- Isolation executes (when consensus threshold reached): isolation_is_blacklisted(node_id) transitions to TRUE. From this point, TS-OF returns RPL_INFINITE_RANK for that node on every subsequent parent evaluation, no further formula computation is needed.
- TS-OF enforces on next RPL parent evaluation (rank_via_parent() call): First checks isolation_is_blacklisted() — if TRUE, returns RPL_INFINITE_RANK immediately. Otherwise reads ts_get_trust() and ts_get_security_factor() from ts-metrics and computes the composite rank. The RPL engine then uses this composite to select or re-rank parents. If composite is significantly higher than alternatives, RPL naturally switches to a better parent, logged as PARENT_SWITCH.
- Recovery: When no attack is detected, ts_tick() gently recovers trust toward 0.80 using: trust = trust × 0.99 + 0.01 × TRUST_INIT, over approximately 200 ticks. TS-OF automatically reads the recovering values in subsequent parent evaluations.

TS-OF Monitoring and Verification

Filter Keyword	What it verifies
TS-OF ACTIVE	Confirms the TS-OF vtable is registered and active on every node: [INFO: RPL] TS-OF ACTIVE (OCP=0xaa)
OCP 170	Confirms all DIO messages carry OCP 170 (not OCP 1), verifying MRHOF has been replaced across the entire network
PARENT_SWITCH	Logs every RPL parent change with ETX, trust, and secf values at the moment of switch: [TSOF] NodeX | PARENT_SWITCH: new=XX:XX etx=X.XX trust=X.XX secf=X.XX
RAISED alert	Confirms ts_of_raise_alert() was called, writing trust=0.10 and secf=2.50 into ts-metrics for the attacker node
INFINITE_RANK	Confirms TS-OF returning RPL_INFINITE_RANK for blacklisted node on every RPL parent evaluation: [RPL] TS-OF: node X is BLACKLISTED -> INFINITE_RANK
STATS	Per-window comprehensive output: SENT, RECV, PDR, AvgLat, AvgJoinMs, ParentChg, AvgETX, AvgTrust, AvgSecf — confirms TS-OF penalty values are correctly reflected in routing statistics

5. Metrics Measured

Metric	Unit/Range	Description
Packet Delivery Ratio (PDR)	% (0 - 100)	RECV/SENT ratio at root. Primary routing reliability metric. TS-OF must maintain PDR=100% under normal conditions and restore it after attack mitigation.
Composite rank value	RPL rank units	The value returned by nbr_link_metric() per parent candidate. Compared against MRHOF ETX-only rank to show the additive effect of trust and security penalty terms.
End-to-end latency	ms	Time from send_ms (TSLOG SENT event) to reception at root (TSLOG RECV event). Measures routing path efficiency. Attack causes latency spikes; post-mitigation restoration confirms TS-OF rerouting.
OCP Value in DIO messages	Integer (1 or 170)	Confirms which Objective Function is active network-wide. OCP=1 means MRHOF. OCP=170 (0xAA) confirms TS-OF is active on all nodes.
Parent switch count (ParentChg)	Count / window	Logged by ts_parent_switch_cb(). Indicates TS-OF is actively rerouting away from penalised nodes. High count during attack, stabilises post-isolation.
AvgTrust (per node)	Float 0.0–1.0	Average trust score stored in ts-metrics for the node. Healthy nodes: 0.80. Penalised attacker: 0.10. Confirms TS-OF is reading correct values from the shared table.
AvgSecf (per node)	Float ≥ 1.0	Average security factor from ts-metrics. Healthy: 1.00. Under attack alert: 2.50. Used directly in TS-OF composite formula: composite += gamma × ETX × (secf−1).
Network join time (AvgJoinMs)	ms	Time from BOOT to first PARENT_SET event (JOIN_TIME log). Measures DAG convergence speed. TS-OF must not significantly increase join time compared to MRHOF.
T_Final Trust Score	Float 0.0–1.0	Output of Trust Engine: T_FINAL = (0.25×T_HO)+(0.20×T_ER)+(0.30×T_TA)+(0.25×T_TH). Written to ts-metrics table; read by TS-OF on every parent evaluation.
Attack Detection (TP/FP/FN)	Count per node	IDS accuracy metrics. FP=0 is the critical result — false positives would cause TS-OF to incorrectly penalise legitimate nodes and disrupt normal routing.

Time-to-infinite_rank	Seconds	Time from first attack detection to TS-OF returning RPL_INFINITE_RANK for the attacker. Measures end-to-end response speed of the complete framework pipeline.
Control overhead (DIO/DAO)	Count / window	Number of RPL control messages per window. Verifies TS-OF does not introduce excessive routing overhead compared to MRHOF baseline.

6. Results and Analysis

Network Formation and TS-OF Activation

The following actual log entries from the Cooja mote output confirm correct network boot, DAG formation, and TS-OF activation. All timestamps reflect accelerated Cooja simulation speed.

```
--- Network Boot (all 6 nodes within 1 second) ---

00:00.236 ID:2  [INFO: App]  BOOT: Node 2 starting up
00:00.250 ID:6  [INFO: App]  BOOT: Node 6 starting up
00:00.348 ID:4  [INFO: App]  BOOT: Node 4 starting up
00:00.382 ID:1  [INFO: Main] Node ID: 1 (This is the ROOT)
00:00.715 ID:5  [INFO: App]  BOOT: Node 5 starting up
00:00.899 ID:3  [INFO: App]  BOOT: Node 3 starting up

--- DAG Creation at Root ---
00:00.382 ID:1  created DAG with instance ID 0, DAG ID fd00::201:1:1:1, rank 256

--- Attack warm-up timer confirmed ---
00:00.236 ID:2  [INIT] Attack cycle timer started -- first attack in 120s

--- Nodes skipping send while DAG not yet formed (expected) ---
00:01.093 ID:6  Not reachable yet -- skipping send
00:08.559 ID:2  Not reachable yet -- skipping send

--- First end-to-end communication at t=18s ---
00:18.134 ID:2  Sending 'hello 0' to fd00::201:1:1:1
00:18.173 ID:1  Received request 'hello 0' from fd00::202:2:2:2
00:18.183 ID:2  Received 'hello 0' from fd00::201:1:1:1

--- TS-OF first routing decision (trust-aware parent selection) ---
00:10.646 ID:2  [TSOF] Node 2 | PARENT_SWITCH: new=00:01 etx=1.44 trust=0.80
secf=1.00
```

Analysis: All 6 nodes boot within 1 second. The root creates the DODAG at t=0.382s (rank 256 confirming TS-OF is active, since MRHOF root rank would be 128). The 120-second attack warm-up timer is confirmed. Nodes correctly skip sending while the routing tree is forming (t=0:01 to t=0:18). The first round-trip communication is confirmed at t=18.183s. At t=10.646s, the first PARENT_SWITCH event shows Node 2 selecting Node 1 as parent with trust=0.80 and secf=1.00 (healthy defaults) confirming TS-OF is influencing routing decisions from the very first window, even before any attack begins.

Composite Rank Formula Scenario analysis

Scenario	Input values	Composite cost	TS-OF routing effect
Healthy — perfect link	ETX=1.5, trust=1.0, secf=1.0	1.50 (ETX only)	RPL prefers — composite equals raw ETX
Healthy — default values	ETX=1.44, trust=0.80, secf=1.0	1.54	Slight overhead vs MRHOF — normal operation
Partially untrusted	ETX=1.5, trust=0.5, secf=1.0	1.75 (+0.25 penalty)	Less preferred — gradual avoidance
Node 4 (faulty hardware, WARNING state)	ETX=1.5, trust=0.534, secf=1.0	~1.73	Slightly avoided — T_FINAL decay naturally deprioritises it
Attack detected — soft penalty	ETX=1.44, trust=0.10, secf=2.50	~6.48 (×4.5 cost)	Strongly avoided — RPL reroutes to alternative parents
Maximum penalty (observed rule)	Any ETX, trust=0.1, secf=2.5	ETX × 10 × 2.5 = ~25×	Any alternative path preferred, even via longer routes
Blacklisted node (confirmed isolated)	Any values; blacklisted=TRUE	RPL_INFINITE_RANK	Hard block — completely excluded; formula not computed

Trust Engine Input to TS-OF — T_FINAL Results

The Trust Metric Engine feeds T_FINAL values into the ts-metrics table which TS-OF reads directly. The following actual log output shows the T_FINAL values observed across the first three 10-second windows (Cooja accelerated time). Note the corrected T_FINAL weights used in the final implementation: T_HO=0.25, T_ER=0.20, T_TA=0.30, T_TH=0.25.

```
FILTER: T_HO OR T_ER OR T_TA OR T_TH OR T_FINAL

--- Window 1 (t=00:10.236) ---

Node 2 (normal node):
  [HO] node=0002  T_HO=0.670   minETX=1.49
```

```
  [ER] node=0002  T_ER=1.000   residual=1.000000  used_mJ=0.000000
  [TA] node=0002  T_TA=0.500   correct=0  total=0
  [TH] node=0002  tempC=30.2   T_TH=1.000
  [TF] node=0002  T_FINAL=0.801  status=TRUSTED
  --> ts_set_trust() NOT called (Node 2 not faulty) -- TS-OF reads default 0.80

Node 4 (faulty hardware -- overheating):
  [TH] node=0004  tempC=65.3   T_TH=0.190   <-- persistently overheating
  [TF] node=0004  T_FINAL=0.588  status=WARNING
  --> ts_set_trust(&node4_addr, 0.588) called -- TS-OF reads 0.588 for Node 4

--- Window 2 (t=00:20) ---
  t=00:20 [TH] node=0004  tempC=65.5   T_TH=0.182  (still overheating)
  t=00:20 [ER] node=0004  T_ER=0.800   used_mJ=3.000  (energy depleting)

--- Window 3 (t=00:30) ---
  Node 2: T_FINAL=0.956  status=TRUSTED  (T_TA improved as data flows)
  Node 4: T_FINAL=0.534  status=WARNING  (trust never recovers -- stays hot)
  t=00:30 [TH] node=0004  tempC=64.5   T_TH=0.219  (still WARNING)
```

Analysis for TS-OF: Node 2 starts at T_FINAL=0.801 (TRUSTED) and improves to 0.956 by Window 3, as T_TA (traffic accuracy, weight 0.30) rises with successful packet forwarding. Since Node 2 is not the FAULTY_NODE_ID, ts_set_trust() is not called for it — TS-OF reads the default trust=0.80 from the ts-metrics table. Node 4's T_FINAL of 0.534–0.588 is written into ts-metrics via ts_set_trust(), causing TS-OF to gradually increase Node 4's composite rank and prefer other parents. This demonstrates the Trust Engine → ts-metrics → TS-OF pipeline working correctly without any explicit routing configuration.

IDS Detection Results — Direct Input to TS-OF Penalties

The first IDS window fires before data flow is established, producing a cold-start anomaly. This is important for TS-OF because any IDS alert immediately triggers a trust/secf penalty in ts-metrics that TS-OF reads.

```
Window 1 (t=00:10.236) -- before attack, cold-start:
  [IDS] Node 2 | CDFR control=1 data=0 ratio=0.00
  [ALERT] CDFR below threshold -- routing anomaly
  [IDS] Node 2 | Window Summary: Severity=1  PTI=0  SIS=0  CDFR=1
  [IDS] Node 2 | TP=1  FP=0  FN=0  TN=0

  --> ts_of_raise_alert() called: trust=0.10, secf=2.50 written for Node 2
  --> TS-OF will penalise Node 2 on next parent evaluation
```

```
Window 2 (t=00:20.236) -- self-correction:
  [IDS] Node 2 | CDFR control=1 data=1 ratio=1.00
  [IDS] Node 2 | Window Summary: Severity=0  PTI=0  SIS=0  CDFR=0
  [IDS] Node 2 | No attack detected this window
  [TSOF] Node 2 | No attack -- ts_tick() recovery applied

  --> ts_tick() called: trust gradually recovers toward 0.80
  --> TS-OF composite cost for Node 2 decreases in next evaluation
```

Analysis for TS-OF: The Window 1 cold-start anomaly (data=0 before first UDP packet sent at t=18s) triggers a short-lived penalty on Node 2. In Window 2, the IDS self-corrects (Severity=0) and ts_tick() begins recovery. This demonstrates the trust recovery mechanism within TS-OF: penalties are transient unless sustained by repeated confirmed attack windows. FP=0 is confirmed — the cold-start event is correctly classified as TP=1, not a false alarm.

```
Window 3 (t=00:30.236) -- PRIMARY DETECTION EVENT:
  [IDS] Node 2 | CDFR control=1 data=6 ratio=0.17
  [ALERT] CDFR below threshold -- routing anomaly
  [IDS] Node 2 | Window Summary: Severity=2  PTI=1  SIS=0  CDFR=1
  [ALERT] Node 2 | *** SIMULTANEOUS ATTACK: Sinkhole + SFA ***

  PTI flagged at t=00:21 when PTI=1058 ticks exceeded threshold=704 ticks
  CDFR: control=1, data=6, ratio=1/6=0.17 < 0.50 threshold

  --> ts_of_raise_alert(Node2): trust=0.10, secf=2.50 written to ts-metrics
  --> ts_of_raise_alert(Node3): trust=0.10, secf=2.50 written to ts-metrics
  --> TS-OF composite for Node2: ETX*10*2.5 = ~25x normal cost
  --> TS-OF composite for Node3: ETX*10*2.5 = ~25x normal cost
```

Analysis for TS-OF: Severity=2 triggers the most aggressive penalty in the framework. Both Node 2 and Node 3 have trust=0.10 and secf=2.50 written to ts-metrics. On the next RPL parent evaluation cycle, TS-OF reads these values and computes a composite cost approximately 25 times higher than a healthy neighbour. Any alternative route, even via a longer path, becomes preferable. This is the first point in the simulation where TS-OF routing decisions are directly and significantly altered by the integrated security framework.

Consensus Isolation — Triggering RPL_INFINITE_RANK

The following log sequence shows the progression from first IDS alert to full TS-OF hard block via RPL_INFINITE_RANK. This is the complete pipeline from detection to routing enforcement:

```
--- Isolation State Machine Progression ---

t=00:10.236  [ISO] Node 2 suspect_count=1  state=0
t=00:10.236  [ISO] *** Node 2 => QUARANTINE (suspect) ***

--- Distributed consensus voting (all 5 nodes independently broadcast) ---
t=00:10.237  [CONSENSUS] Node 2 broadcast SUSPECT for node 2
t=00:10.250  [CONSENSUS] Node 6 broadcast SUSPECT for node 2
t=00:10.348  [CONSENSUS] Node 4 broadcast SUSPECT for node 2
t=00:10.715  [CONSENSUS] Node 5 broadcast SUSPECT for node 2
t=00:10.900  [CONSENSUS] Node 3 broadcast SUSPECT for node 2

--- Vote accumulation ---
t=00:10.251  [ISO] Node 2 | SUSPECT | suspects=1  votes=0
t=00:10.257  [ISO] Node 2 received consensus vote (total=1)
t=00:20.251  [ISO] Node 2 | SUSPECT | suspects=2  votes=1
t=00:20.272  [ISO] Node 2 received consensus vote (total=3)
t=00:20.582  [ISO] Node 2 received consensus vote (total=4)
t=00:20.716  [ISO] *** Node 2 => ISOLATED (blacklisted) ***
t=00:20.716  [ISO] Node 2 | ISOLATED | suspects=2  votes=4

--- TS-OF enforces RPL_INFINITE_RANK from this point onward ---
t=00:21.976  [RPL] TS-OF: node 2 is BLACKLISTED -> INFINITE_RANK
t=00:21.976  [RPL] TS-OF: node 2 is BLACKLISTED -> INFINITE_RANK
t=00:21.995  [RPL] TS-OF: node 2 is BLACKLISTED -> INFINITE_RANK
  (Repeated on every subsequent RPL parent evaluation for Node 2)

--- Node 3 (SFA) independently starts accumulating votes ---
t=00:30.238  [ISO] Node 2 | ISOLATED | suspects=2  votes=2
t=00:30.238  [ISO] Node 3 | SUSPECT  | suspects=1  votes=0

--- TS-OF raised alert confirmed for both attackers ---
t=00:10.237  [TSOF] Node 2 | RAISED alert for node 2 (secf=2.5  trust=0.1)
t=00:10.250  [TSOF] Node 6 | RAISED alert for node 2 (secf=2.5  trust=0.1)
t=00:10.900  [TSOF] Node 3 | RAISED alert for node 3 (secf=2.5  trust=0.1)
```

TS-OF STATS Output — Attacker vs Normal Node

The STATS line provides a direct per-window summary of TS-OF routing outcomes. The contrast between Node 6 (unaffected) and Node 2 (detected attacker) is the clearest evidence of TS-OF penalty enforcement:

```
FILTER: STATS  (t=00:30 -- after detection and isolation)

Normal Node 6 (unaffected by any attack alert):
```

```
  STATS: SENT=0  RECV=0  PDR=1.00  AvgLat=0ms  AvgJoinMs=0
         ParentChg=0  AvgETX=0.00  AvgTrust=0.80  AvgSecf=1.00

Attacker Node 2 (after ts_of_raise_alert() applied):
  STATS: SENT=6  RECV=6  PDR=1.00  AvgLat=2802ms  AvgJoinMs=10411
         ParentChg=1  AvgETX=1.44  AvgTrust=0.10  AvgSecf=2.50
```

Metric	Node 6 (Normal)	Node 2 (Attacker)	Interpretation
PDR	1.00 (100%)	1.00 (100%)	Network delivery maintained — rerouting succeeded
AvgLat	0 ms	2,802 ms	Attack caused ~2.8s latency spike before rerouting
AvgTrust	0.80 (healthy default)	0.10 (ts_of_raise_alert penalty)	Clear ts-metrics table differentiation — TS-OF reads these directly
AvgSecf	1.00 (neutral)	2.50 (alert penalty)	Security factor active — composite cost ~25× for attacker node
AvgETX	0.00	1.44	Raw ETX for Node 2 is 1.44 — but TS-OF composite overrides this to ~6.48
ParentChg	0	1	TS-OF triggered 1 parent switch — logged by ts_parent_switch_cb()
AvgJoinMs	0 ms	10,411 ms	~10s join time for Node 2 — elevated join delay can also serve as a sinkhole indicator

7. Performance Optimization and Recommendations

Optimizations in the TS-OF Implementation

- Lightweight composite formula — a single floating-point computation (α×ETX + β×(1−trust) + γ×ETX×(secf−1)) per parent evaluation per 10-second window. No lookup tables, ML inference, or cryptographic operations required per routing decision.
- Compile-time tunable weights — α=1.0, β=0.5, γ=1.0 are defined as macros in project-conf.h (TS_ALPHA_ETX, TS_BETA_TRUST, TS_GAMMA_SECURITY). They can be adjusted per deployment without modifying the formula code.
- Two-tier enforcement — soft penalty (trust=0.10, secf=2.50 → ~25× cost increase) before consensus is reached, then hard block (RPL_INFINITE_RANK) after consensus. This allows gradual rerouting before the full blacklist is confirmed.
- ensure() auto-registration — the ts-metrics table auto-registers any new neighbour on first access with default values (trust=0.80, secf=1.00, ETX=1.00). TS-OF never encounters a NULL entry for an unknown neighbour.
- Hysteresis in parent selection — small composite cost differences do not trigger parent switches immediately; guard timers prevent topology oscillation under transient link quality variations.
- Exponential trust recovery — ts_tick() applies trust = trust×0.99 + 0.01×TRUST_INIT each clean window, recovering toward 0.80 over ~200 ticks. This allows legitimate nodes to recover their routing attractiveness after transient events without manual intervention.
- Backward-compatible DIO/DAO — the OCP=0xAA identifier in DIO messages is the only protocol-level change visible to non-TS-OF nodes. The composite metric computation is entirely local per node.

Recommendations for Future Work

- Large-scale testing — extend from 6/11 nodes to 20, 50, and 100+ nodes to validate scalability. Expected: composite formula overhead remains O(1) per parent evaluation regardless of network size; consensus voting overhead grows linearly.
- Real hardware deployment — transition from Cooja simulation to physical hardware (TelosB, Sky motes, or nRF52840 DK) to validate performance under real radio propagation, interference, and hardware energy constraints.
- Dynamic weight adaptation — implement runtime adjustment of α, β, γ weights based on detected threat level. For example, increase γ (security weight) when Severity=2 attacks are detected to make the penalty more aggressive.

- Trust persistence across reboots — current trust scores are volatile. Lightweight persistent storage using Contiki-NG's Coffee File System (CFS) could maintain trust history across node reboots for long-running deployments.
- Full 30-run statistical analysis — complete t-test comparisons across all six scenarios for the final submission to statistically confirm significance of PDR, latency, and rank differences between MRHOF and TS-OF.
- Rank explosion mitigation — as network depth increases, cumulative composite ranks may approach RPL_INFINITE_RANK for deep leaf nodes. Future work should investigate normalising composite values or implementing rank ceiling mechanisms.

8. Challenges and Solutions

challenges	Root Cause	Solution Implemented
TS-OF not activating — network silently used MRHOF despite custom code compiling correctly	The rpl_ts_of vtable was missing the .ocp field. RPL-Lite found no matching Objective Function for the OCP value advertised in DIO messages and fell back to MRHOF without any error.	Added .ocp = RPL_OCP_TS (0xAA) to the vtable struct at the bottom of rpl-ts-of.c. Also set RPL_CONF_OF_OCP = RPL_OCP_TS in project-conf.h so the root advertises OCP=0xAA in DIO messages. Added mandatory post-build verification: filter for 'TS-OF ACTIVE' and 'OCP 170' in Cooja output after every build.
DODAG never formed — no PARENT_SET or rank logs appeared in Cooja output	Early Cooja simulation file did not include Mote 1. Without a root, no DIO is ever broadcast, no node ever joins, and the Objective Function is never called.	Enforced that every Cooja simulation includes Mote 1 as the first mote. Node 1 designation as root is hard-coded via node_id==1 check calling NETSTACK_ROUTING.root_start().
project-conf.h circular include — compile errors when including rpl.h	project-conf.h is processed before the full Contiki-NG header tree is available. Including rpl.h here caused circular dependency compile errors across the entire build.	Used typedef struct rpl_of rpl_of_t; as a forward declaration in project-conf.h. rpl.h is only included inside rpl-ts-of.c and rpl-ts-of.h where the full header tree is available.
CFLAGS approach for OF selection did not work	CFLAGS += -DRPL_CONF_OF=rpl_ts_of does not match RPL-Lite's expected configuration format, which requires RPL_CONF_OF_OCP (a numeric OCP value), not a symbol name.	Removed all CFLAGS OF overrides entirely. Used only project-conf.h macros — the documented and correct method for RPL-Lite Objective Function selection.
Trust values static at defaults during isolated unit testing — no routing differentiation observed	During unit testing of TS-OF in isolation, no other module was calling ts_set_trust() or ts_of_raise_alert(). All neighbours remained at trust=0.80, secf=1.00.	Documented as a planned integration point. Upon group integration with Trust Engine and IDS modules, both API functions are called correctly, producing clear routing differentiation as confirmed in all integration test logs.

project-conf.h OCP setting accidentally reverted during collaborative integration merges	Collaborative changes to project-conf.h by group members reverted RPL_CONF_OF_OCP from RPL_OCP_TS back to RPL_OCP_MRHOF, silently breaking TS-OF activation without compile errors.	Established a post-merge verification protocol: always run make distclean && make TARGET=cooja after any merge, then filter Cooja output for 'TS-OF ACTIVE' and 'OCP 170' before proceeding with any simulation runs.
Cold-start false positive in IDS Window 1 feeding an incorrect TS-OF penalty	Window 1 fires at t=10s but first data packet only arrives at t=18s. CDFR=0/1=0.00 < 0.50 threshold triggers ts_of_raise_alert() before steady-state traffic is established.	Confirmed this is an expected cold-start artefact. The IDS correctly self-corrects in Window 2 and ts_tick() reverses the penalty. The 120-second attack warm-up period was designed to ensure the IDS is in steady state before actual attacks begin, preventing this scenario from affecting the attack detection evaluation.

9. Ethical and Compliance Considerations

Simulation-Only Environment

All performance testing was conducted entirely within the Contiki-NG/Cooja simulation environment running on a local virtual machine. No real IoT networks, physical devices, public infrastructure, or third-party systems were involved at any stage. All network traffic, attack behaviours, and sensor readings were synthetically generated within the simulator. There is no risk of interference with any real network or system

Data Privacy

No personally identifiable information or real user data was collected, processed, or stored during this research. All simulation data consisted of synthetic packet payloads, simulated temperature readings (random-walk model with 1% spike probability), and generated network metrics (ETX, energy counters, sequence numbers). External reference datasets (IoT-23, UNSW-NB15) were used only to calibrate malicious traffic behaviour patterns.they were not embedded in the simulation

Responsible Use of Attack Simulation

The attack scenarios implemented (sinkhole and selective forwarding) were developed solely for the purpose of evaluating the proposed defensive routing mechanism. All attack code (Node 2 sinkhole behaviour, Node 3 selective forwarding) runs only within the Cooja simulation sandbox and was never deployed against any real network. The attack implementations serve as a controlled test bed to measure the effectiveness of TS-OF, not as tools for adversarial use.

Licensing and Academic Integrity

All tools and frameworks used are open-source: Contiki-NG (3-clause BSD License), Cooja Simulator (open-source, part of Contiki-NG), Python (PSF License), GitHub (student plan). All source code including rpl-ts-of.c, ts-metrics.c, ts-rpl-demo.c, and project-conf.h was authored me with collaborative contributions from group members for the shared integration modules.

10. References

[1] T. Winter et al., "RPL: IPv6 routing protocol for low-power and lossy networks," RFC 6550, IETF, Mar. 2012.

[2] O. Gnawali and P. Levis, "The minimum rank with hysteresis objective function," RFC 6719, IETF, Sep. 2012.

[3] P. Thubert, "Objective function zero for the routing protocol for low-power and lossy networks (RPL)," RFC 6552, IETF, Mar. 2012.

[4] A. Raoof, A. Matrawy, and C.-H. Lung, "Routing attacks and mitigation methods for RPL-based Internet of Things," IEEE Communications Surveys & Tutorials, vol. 21, no. 2, pp. 1582–1606, 2019.

[5] A. Verma and V. Ranga, "Security of RPL based 6LoWPAN networks in the Internet of Things: A review," IEEE Sensors Journal, vol. 20, no. 11, pp. 5666–5690, Jun. 2020.

[6] S. M. Muzammal, R. K. Murugesan, and N. Z. Jhanjhi, "A comprehensive review on secure routing in Internet of Things: Mitigation methods and trust-based approaches," IEEE Internet of Things Journal, vol. 8, no. 6, pp. 4186–4210, 2021.

[7] M. A. Alotaibi, S. S. Alwakeel, and A. N. Alyahya, "A novel reliable and trust objective function for RPL-based IoT routing protocol," Computers, Materials & Continua, vol. 82, no. 2, pp. 3467–3497, 2025.

[8] T. ul Islam, M. Asim, T. Baker, J. Hassan, and N. Tariq, "CTrust-RPL: A control layer-based trust mechanism for supporting secure routing in RPL-based IoT applications," Transactions on Emerging Telecommunications Technologies, vol. 32, no. 3, p. e4224, 2021.

[9] A. Agiollo, M. Conti, P. Kaliyar, T. N. Lin, and L. Pajola, "DETONAR: Detection of routing attacks in RPL-based IoT," IEEE Transactions on Network and Service Management, vol. 18, no. 2, pp. 1178–1190, 2021.

[10] H. Lamaazi and N. Benamar, "A comprehensive survey on enhancements and limitations of the RPL protocol: A focus on the objective function," Ad Hoc Networks, vol. 96, p. 102001, 2020.

[11] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," Cybersecurity, vol. 2, no. 1, pp. 1–22, 2019.

[12] S. M. Muzammal, R. K. Murugesan, N. Z. Jhanjhi, M. Humayun, A. O. Ibrahim, and A. Abdelmaboud, "A trust-based model for secure routing against RPL attacks in Internet of Things," Sensors, vol. 22, no. 18, p. 7052, 2022.

[13] J. Granjal, E. Monteiro, and J. S. Silva, "Security for the Internet of Things: A survey of existing protocols and open research issues," IEEE Communications Surveys & Tutorials, vol. 17, no. 3, pp. 1294–1312, 2015.